GEMM

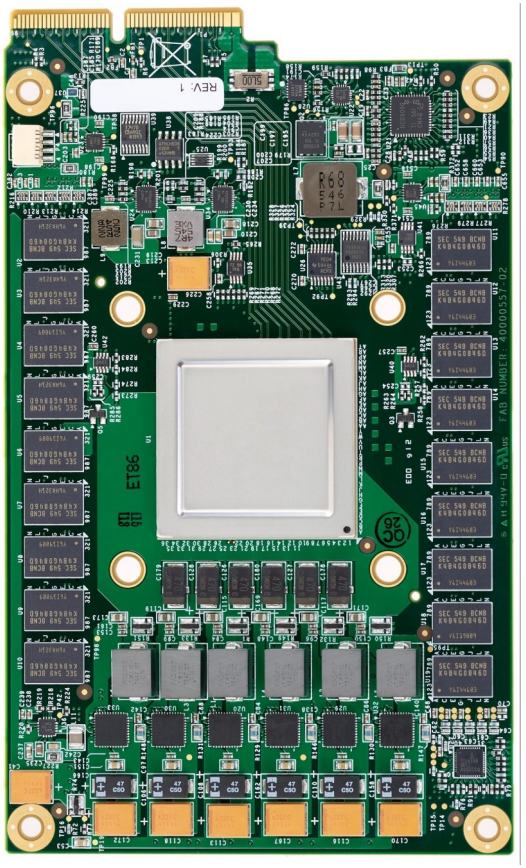
Team 8

Adam Ali, Sne Samal,
Yueming Wang, Zian Lin

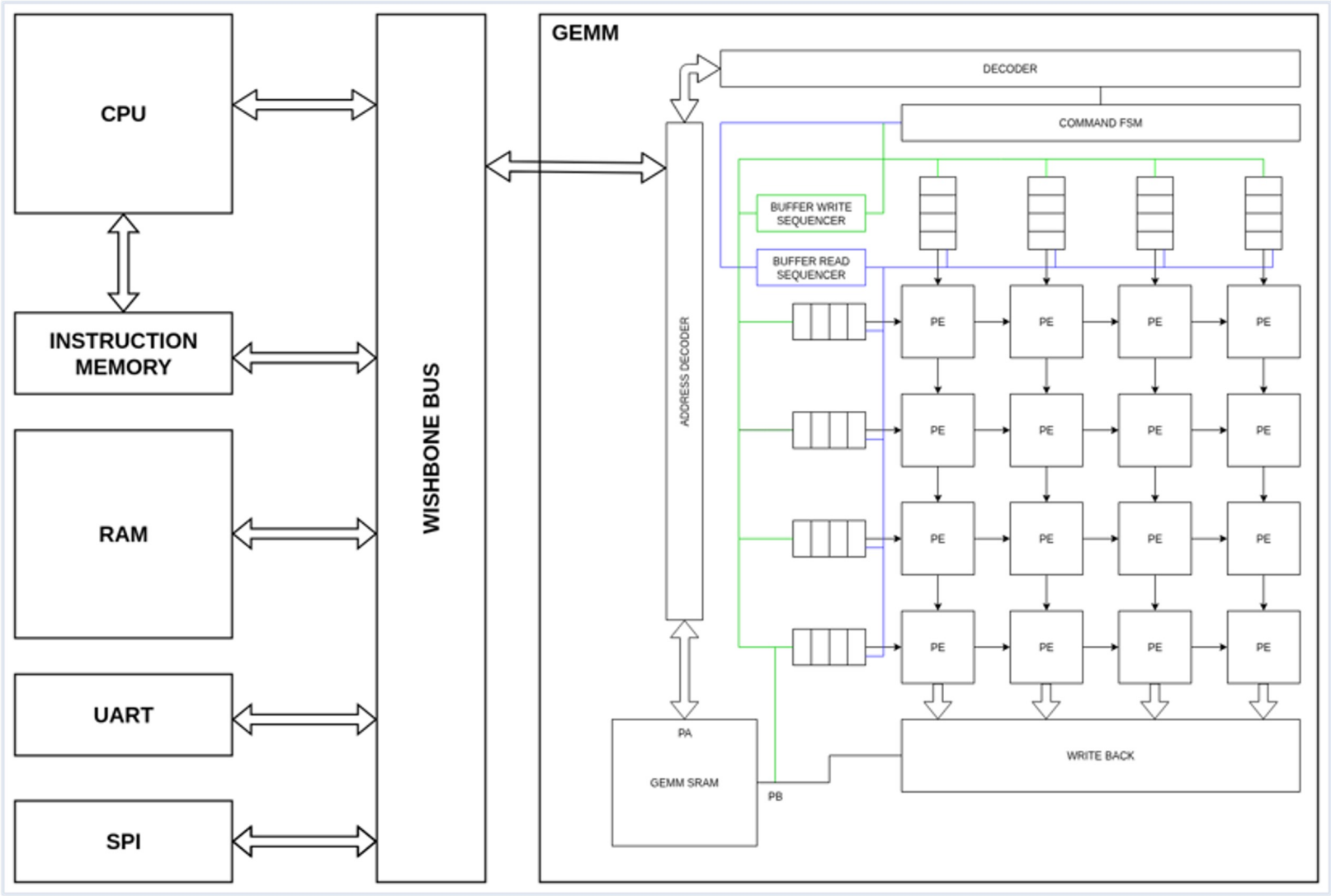
18/06/2026

Background & Motivation

- GEneral Matrix to Matrix multiplication is the core kernel behind machine learning, computer graphics, and signal processing
- Defined as C = αAB+βC
- We perform GEMM in silicon using a 4 by 4 systolic array
- Systolic arrays allow us to keep data flowing locally, maximize operand reuse, and packs parallelism into a small, layout-friendly structure
- To support matrices larger than the hardware array dimensions, tiling and workload orchestration are handled in C firmware running on a RISC-V processor,



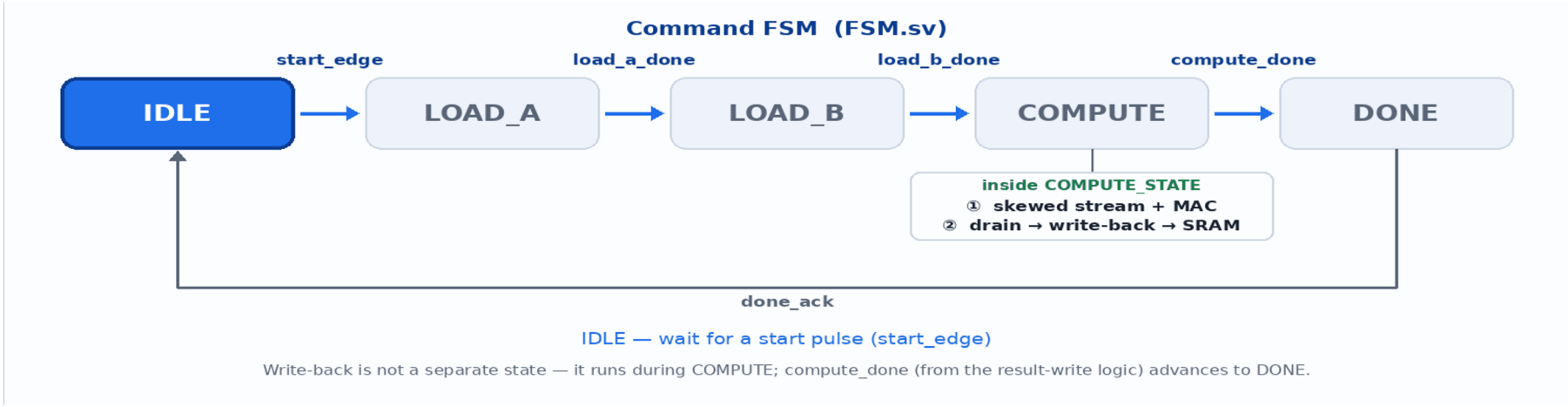
Google TPUv1 (2015) features a 256x256 systolic array of 8-bit integer multiply-accumulate units, delivering 92 teraops/s


CPU
INSTRUCTION MEMORY
RAM
UART
SPI
WISHBONE BUS
GEMM
DECODER
COMMAND FSM
BUFFER WRITE SEQUENCER
BUFFER READ SEQUENCER
ADDRESS DECODER
PE
PA
GEMM SRAM
PB
WRITE BACK

ISA + Memory Map

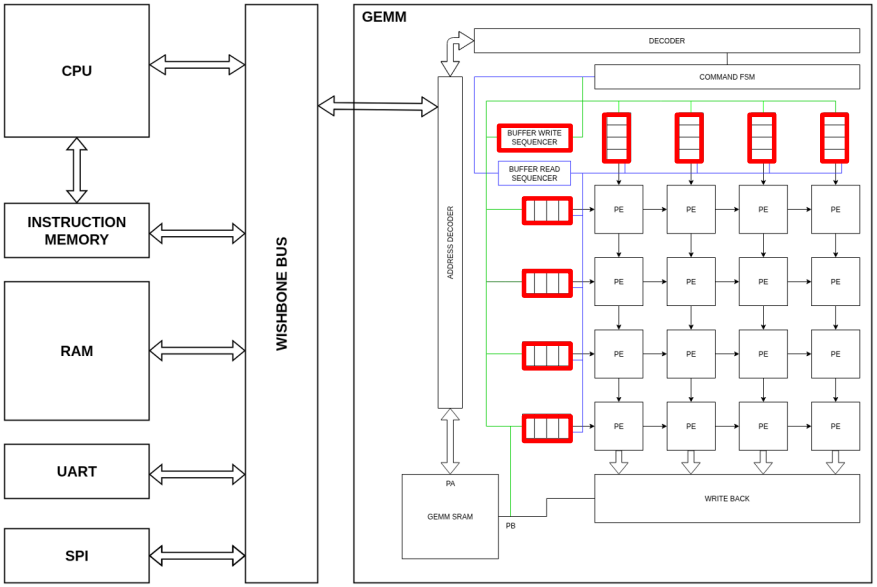
Instruction	Function
`MMINT8 a b c`	Matrix multiply; a/b/c = west / north / result base addresses
`TILE_BEGIN`	Preserve systolic array contents (start of tile)
`TILE_END`	Clear systolic array contents (end of tile)

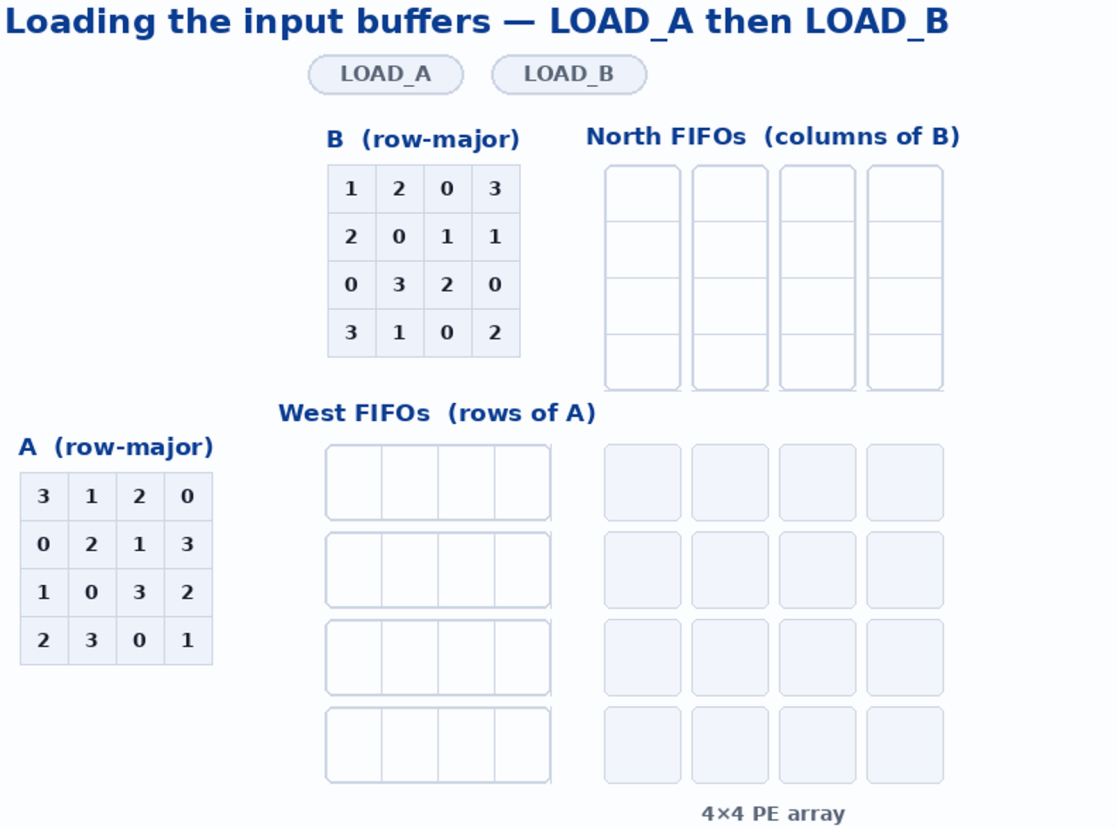
Offset	Register	Access	Purpose
`+0x0`	`INSTRUCTION_REG`	R/W	Issue instructions
`+0x4`	`STATUS_REG`	R	Computing / idle
`+0x8–0x87`	SRAM window	R/W	32 × 32-bit words (128 B)



LOAD

- Triggered by the command FSM
- Buffer Write Sequencer (BWS) streams operands out of the SRAM and into FIFOs
- West side buffers feed matrix A row-by-row, North side feed B across all 4 columns at once.
- Each is a 4-deep x 8-bit FIFO

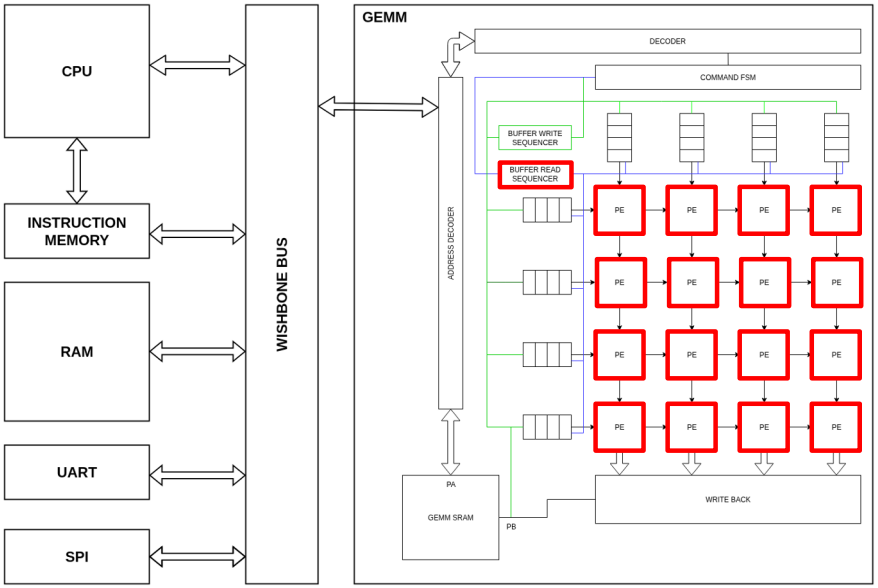


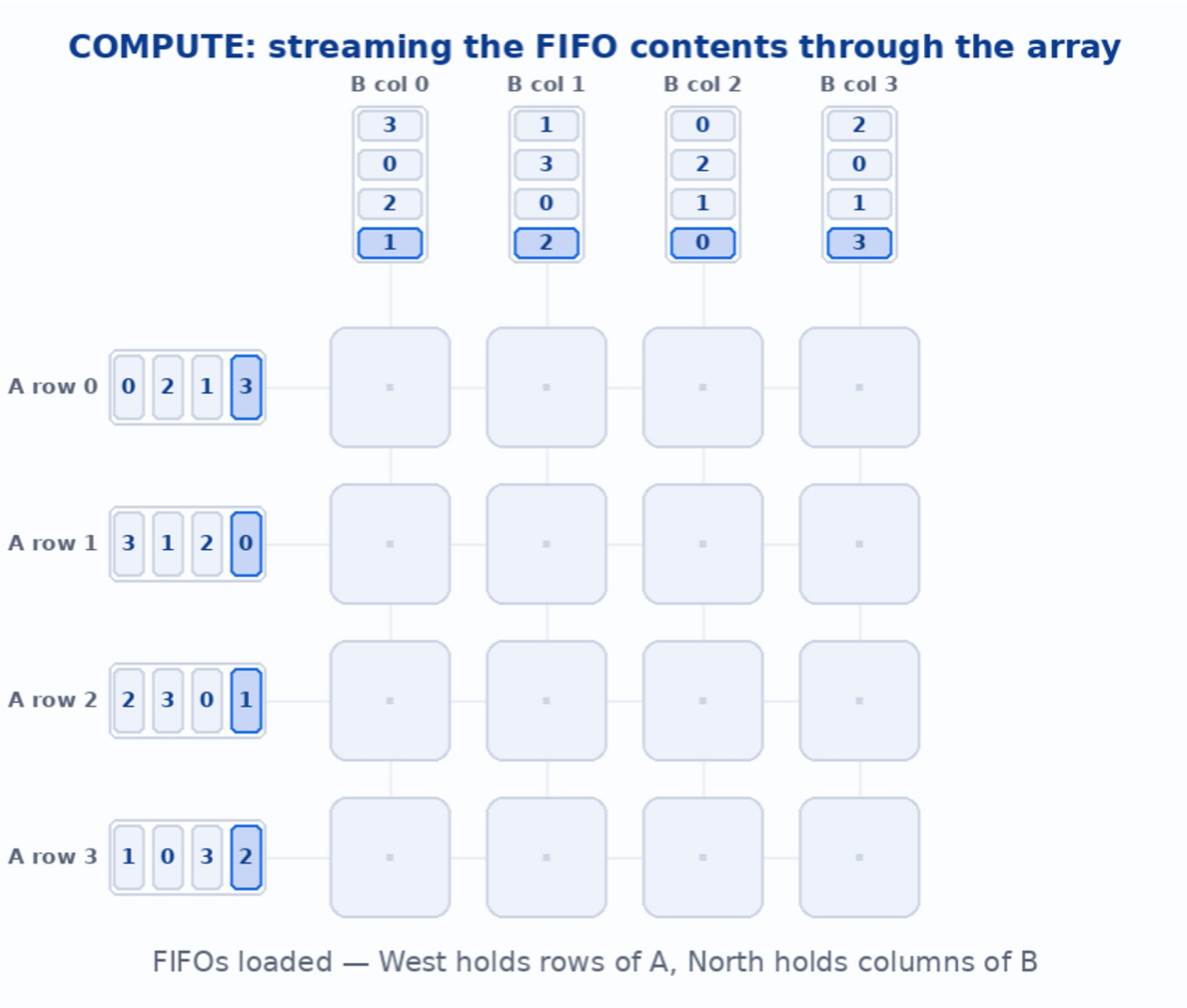


Operands sit in SRAM row-major → loaded into the West (A) and North (B) FIFOs

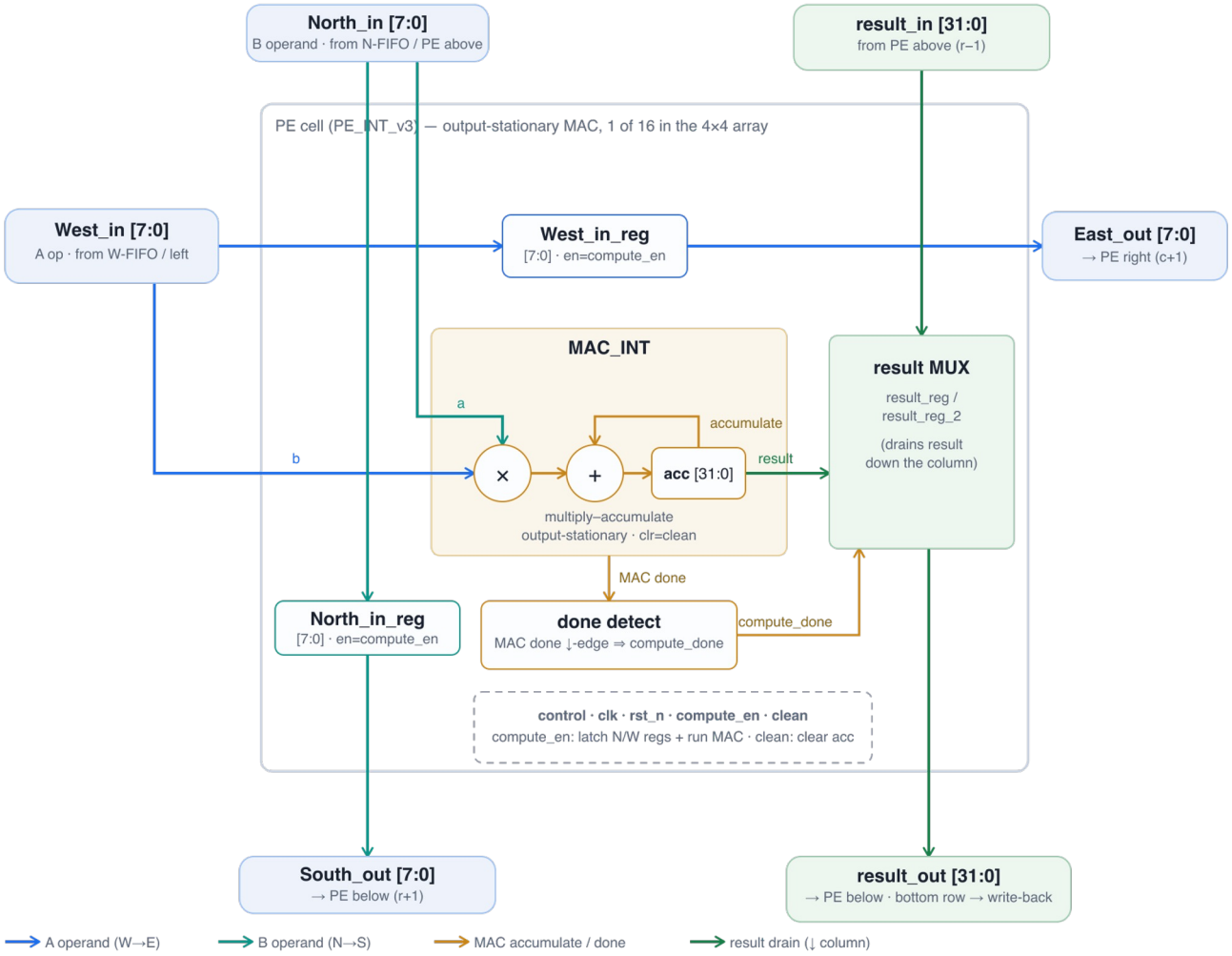
COMPUTE

- Buffer Read Sequencer (BRS) drains the input FIFOs in a skewed wavefront.
- Each PE holds a MAC unit - a signed 8 by 8 multiply accumulated into a 32 bit register.
- Valid bits propagate with the data so that PE only start multiplying once real operands reach it





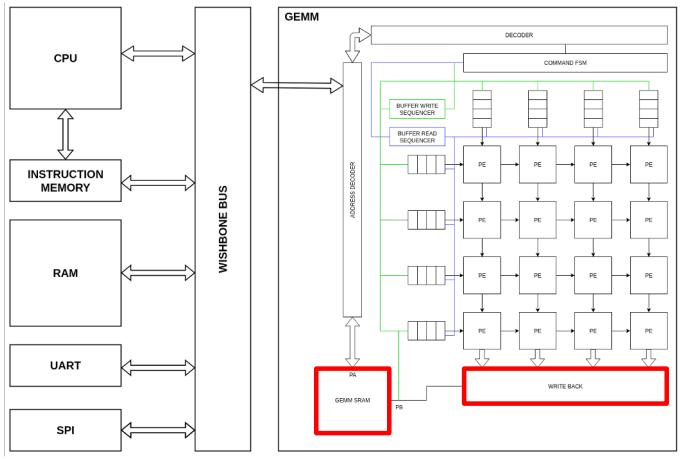
PE design

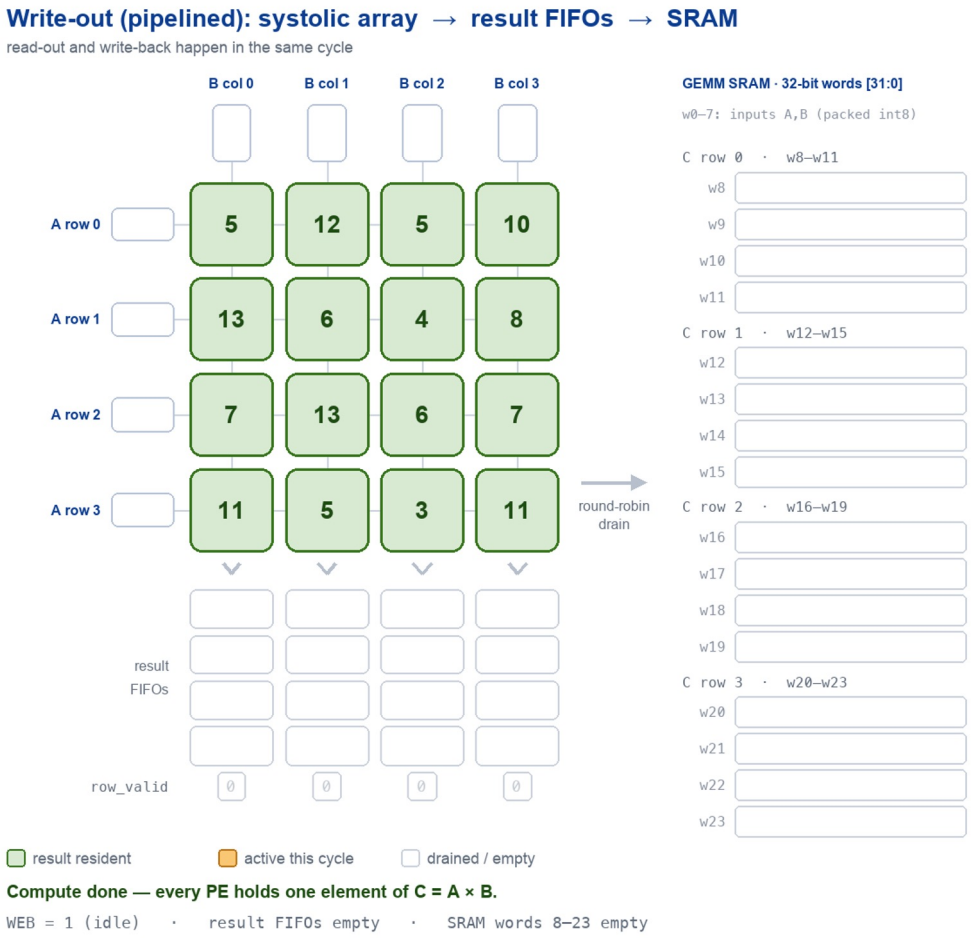

North_in [7:0]
B operand · from N-FIFO / PE above
result_in [31:0]
from PE above (r−1)
PE cell (PE_INT_v3) — output-stationary MAC, 1 of 16 in the 4×4 array
West_in [7:0]
A op · from W-FIFO / left
West_in_reg
[7:0] · en=compute_en
East_out [7:0]
→ PE right (c+1)
MAC_INT
a
b
accumulate
×
+
acc [31:0]
result
multiply–accumulate
output-stationary · clr=clean
result MUX
result_reg /
result_reg_2
(drains result
down the column)
MAC done
North_in_reg
[7:0] · en=compute_en
done detect
MAC done ↓-edge ⇒ compute_done
compute_done
control · clk · rst_n · compute_en · clean
compute_en: latch N/W regs + run MAC · clean: clear acc
South_out [7:0]
→ PE below (r+1)
result_out [31:0]
→ PE below · bottom row → write-back
A operand (W→E)
B operand (N→S)
MAC accumulate / done
result drain (↓ column)

WRITE-BACK

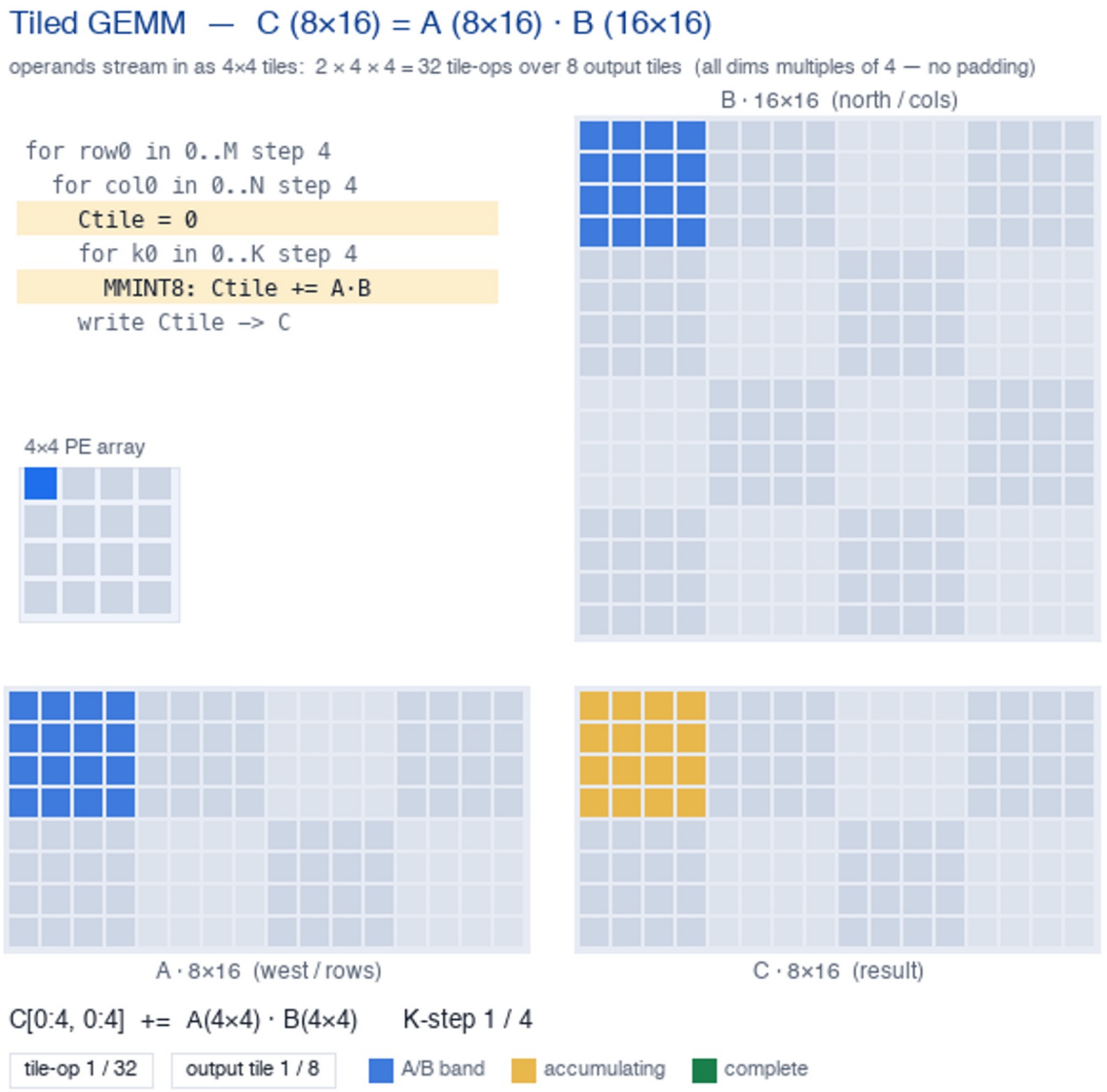
fully pipelined, zero-stall result drain

- One FIFO per column catches the array's anti-diagonal output (≤ 4 results/cycle)
- FIFOs decouple array ↔️ SRAM (1 × 32-bit word/cycle) → PEs never stall
- Read-out and write-back fully overlap — pipelined, not two phases
- Round-robin drain rebuilds C in row-major order, for free (no transpose)
- DONE auto-detected when FIFOs drain → CPU polls status, reads SRAM (no IRQ/DMA)





Software Tiling



BSP / HAL API — UART & GEMM Accelerator

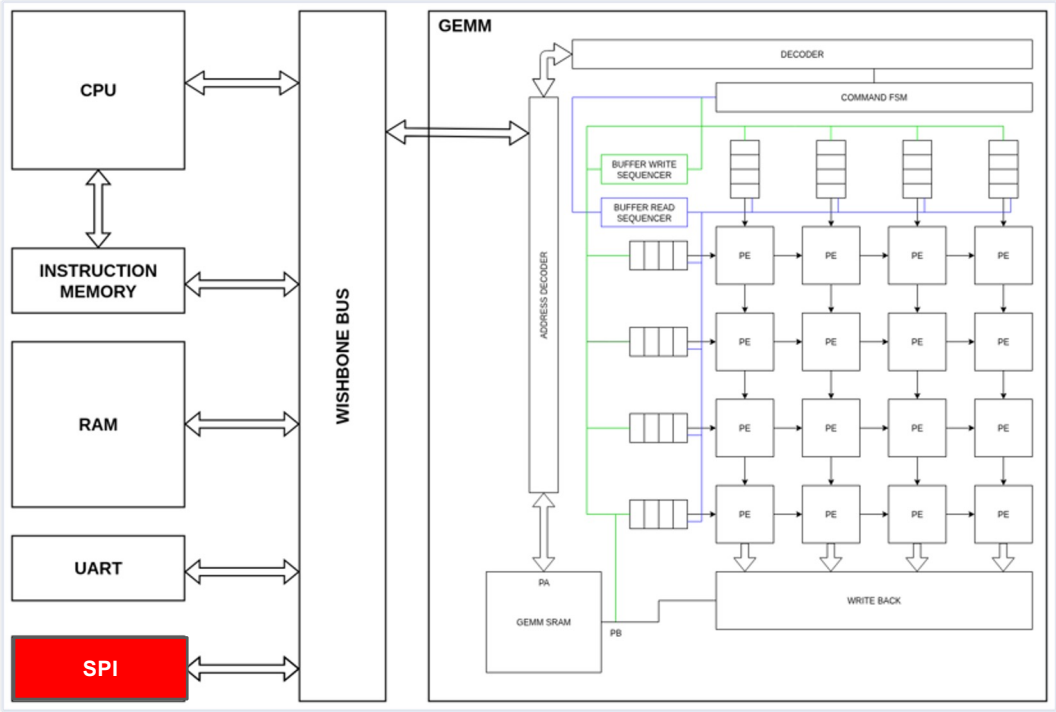
Board-support functions for driving the on-chip UART and the GEMM matrix-multiply IP (mono = function, right = behaviour)

UART HAL	
`uart_putc(int c)`	Transmit one byte; blocks while the UART TX is busy
`uart_getc(void)`	Receive one byte; blocks until RX is ready (returns the byte)
`uart_send_u32(int v)`	Transmit a 32-bit word as 4 bytes, little-endian (LSB first)
`uart_recv_u32(void)`	Receive a 32-bit word from 4 bytes, little-endian (LSB first)
GEMM HAL — high-level	
`gemm_init(void)`	Reset / arm the accelerator before a run
`gemm_run(A, a_words, B, b_words, C, c_words, V)`	One 4×4 tile, end-to-end: load A,B → MMINT8 → wait → read C
`gemm_run_auto_tile_int8(A, U, V, B, W, C)`	Arbitrary-size INT8→INT32 GEMM: auto 4×4 tiling + K-accumulation
GEMM HAL — control & status	
`gemm_issue_tile_begin(void)`	Issue TILE_BEGIN — open a tile, preserve the PE accumulators
`gemm_issue_mmint8(ly)`	Issue MMINT8 a b c — start a 4×4 matmul from the SRAM operands
`gemm_issue_tile_end(void)`	Issue TILE_END — acknowledge DONE and clear the array
`gemm_wait_done(void)`	Block (spin) until the STATUS register reads DONE
`gemm_status() / gemm_is_done() / gemm_is_busy()`	Read the command-FSM state and derive done / busy flags
GEMM HAL — operand / result memory	
`gemm_load_A_words(ly,…) / gemm_load_B_words(ly,…)`	Write packed A / B operands into the GEMM SRAM
`gemm_read_C_words(ly,…)`	Read the INT32 result tile back from the GEMM SRAM
`gemm_sram_write_words() / gemm_sram_read_words()`	Raw word-granular GEMM-SRAM read / write

Internal helpers used by gemm_run_auto_tile_int8: gemm_layout · pack_int8_to_words · extract_A/B_tile_int8 · scatter_C_tile_int32 · gemm_pack_instr / gemm_make_*_instr · run_one_tile

Why SPI?

For peripheral communication, debug, and demonstrating a complete memory-mapped control-oriented IP block.



SPI microarchitecture

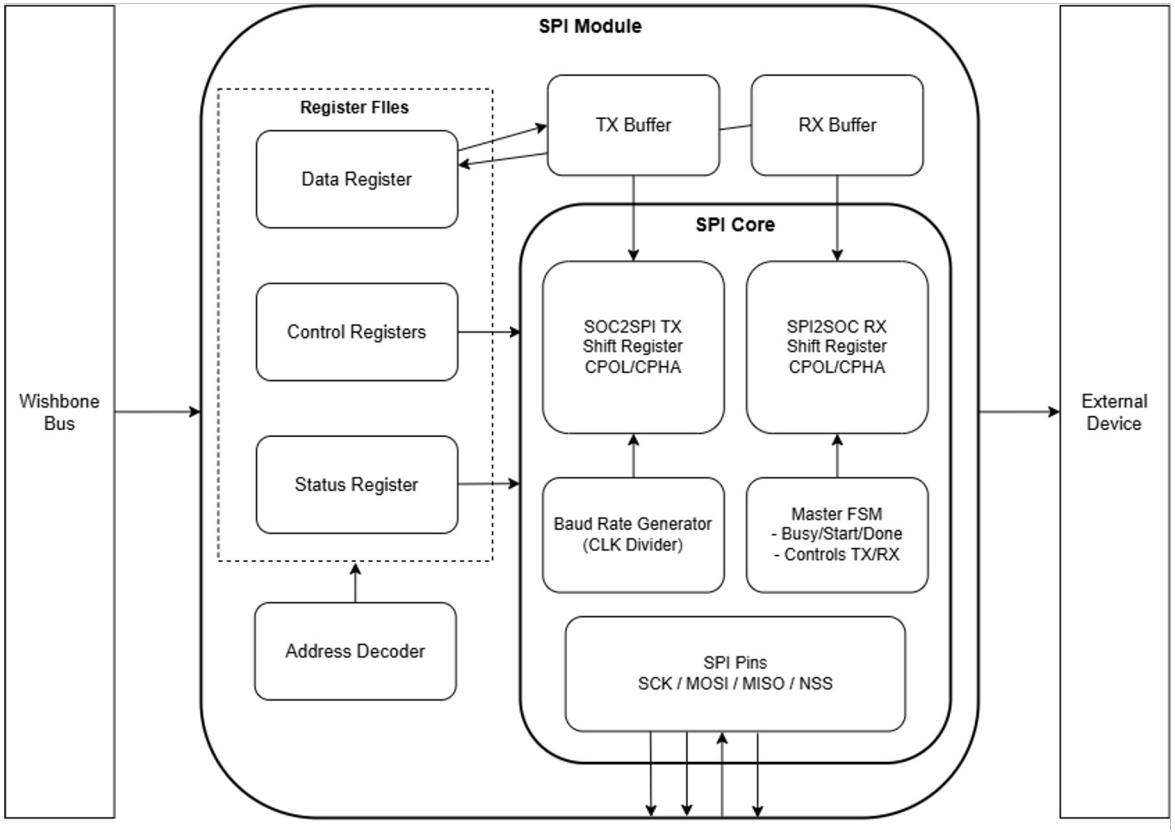
Dual Path Design

Bus Mode:

Traditional SPI workflow
CPU → TX FIFO → SPI Core → RX FIFO → CPU

Loader Mode:

Send dummy byte to generate SCK
Converts every valid MISO byte into a one-cycle SRAM byte-write



SPI core

- Shift register
- Master FSM
- CPOL/CPHA logic
- Baud-rate generator
- TX/RX buffers

Serial pin drivers

- MOSI: drive data out
- MISO: sample data in
- SCK: generated clock
- NSS: chip select

SPI Register Model

Register	Purpose	Notes
SPI_CR1	Enable, master mode, CPOL, CPHA, bit order, divider	Main configuration register
SPI_CR2	Register for expansion	Reserved for interrupts/DMA
SPI_SR	Status flags: TXE, RXNE, BSY	Software observes buffer state and active transfer
SPI_DR	Transmit / receive data register	Write to transmit path; read received data

Status flags

TXE Transmit buffer available

RXNE Receive data available

BSY Active serial transaction

Logic Synthesis Flow

Physical Implementation

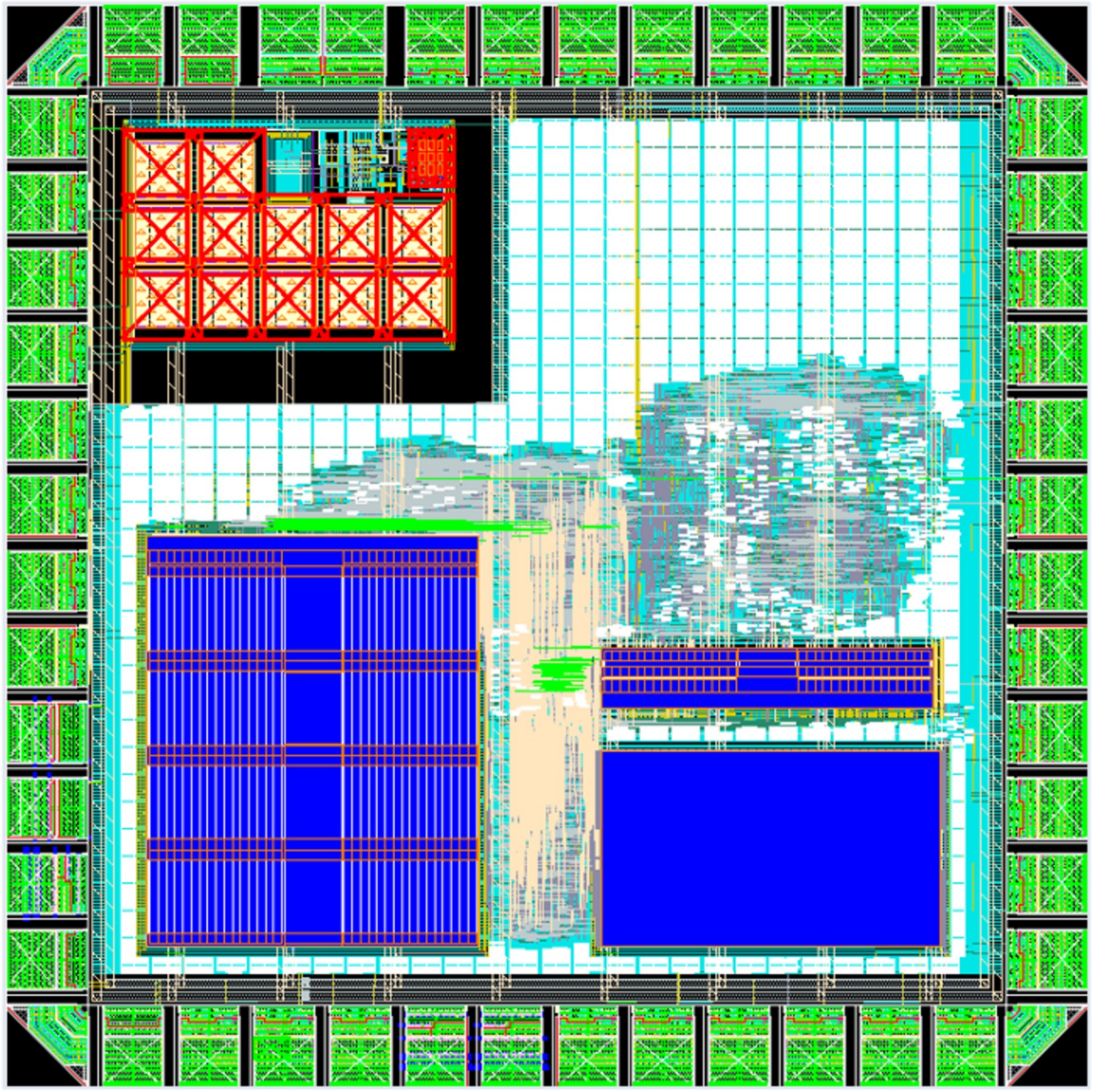
Analog / digital isolation

CPU to SRAM Floorplanning

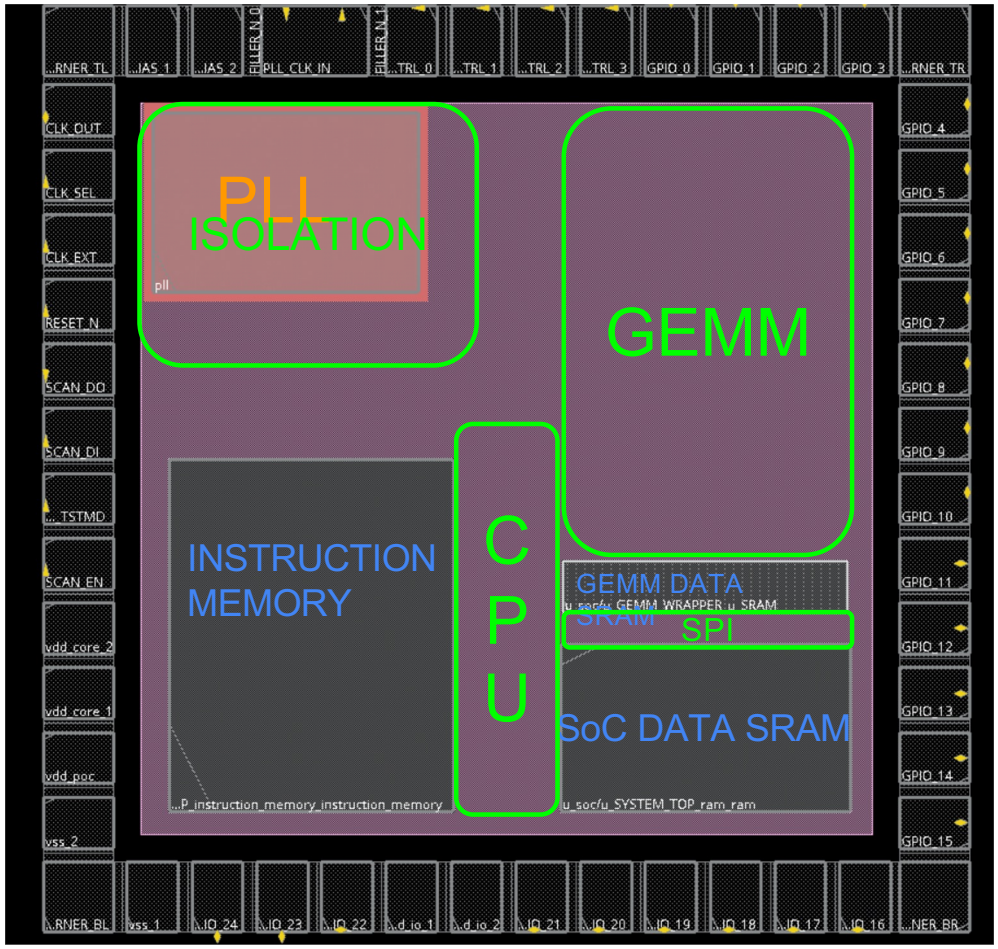
Dedicated clock-tree corridor for cleaner CTS and lower skew

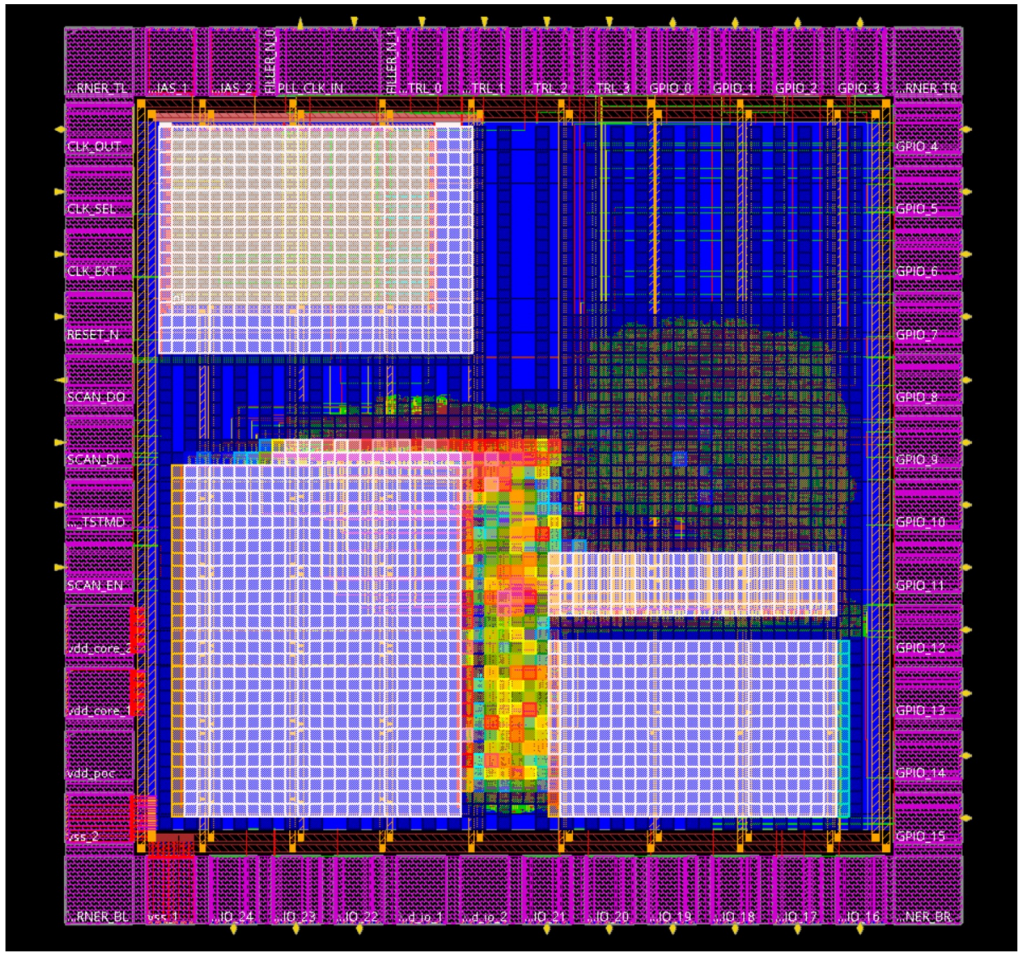
Macro placement to reduce routing congestion

Power integrity and routability awareness

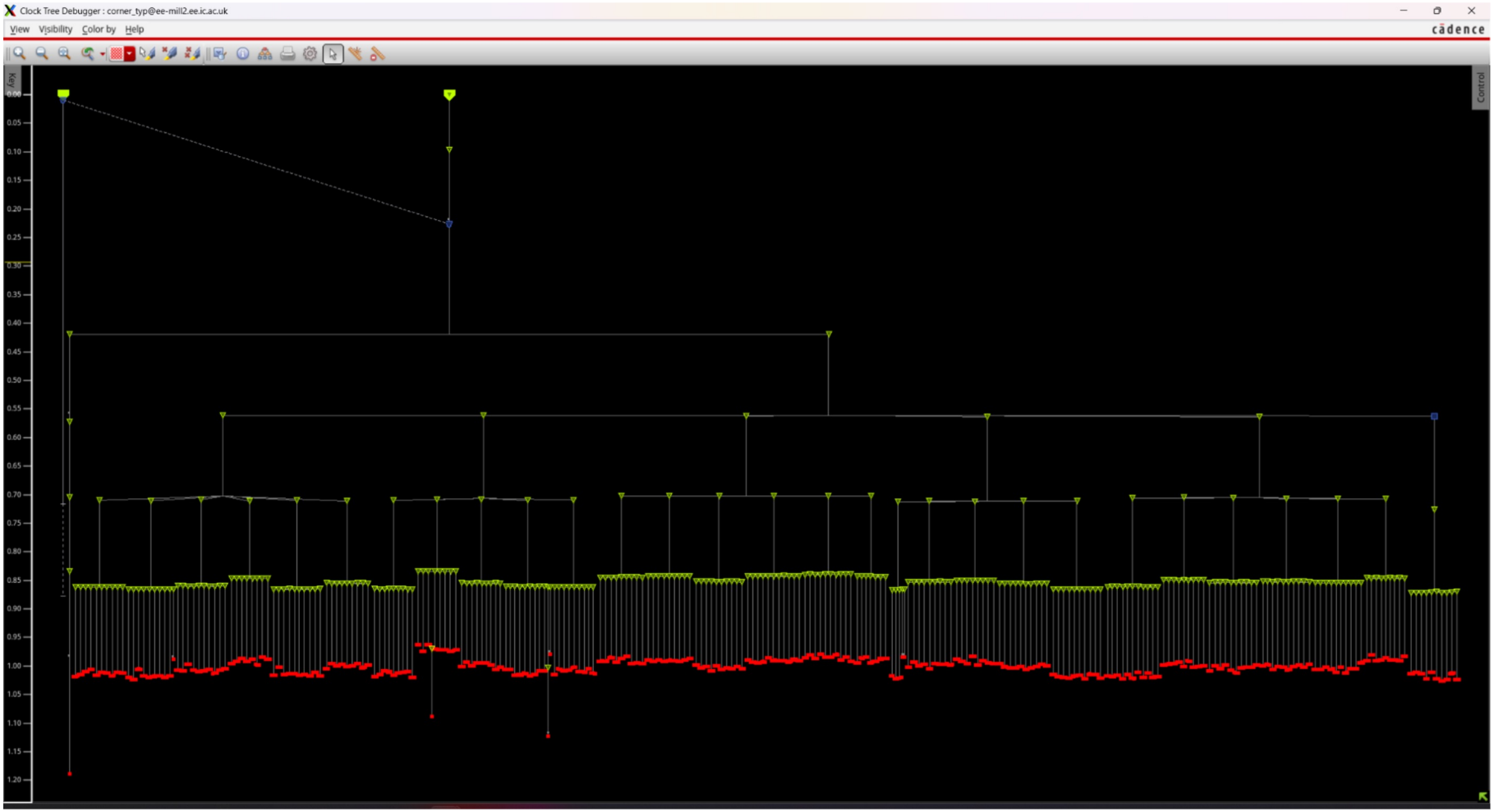


Macro Placement



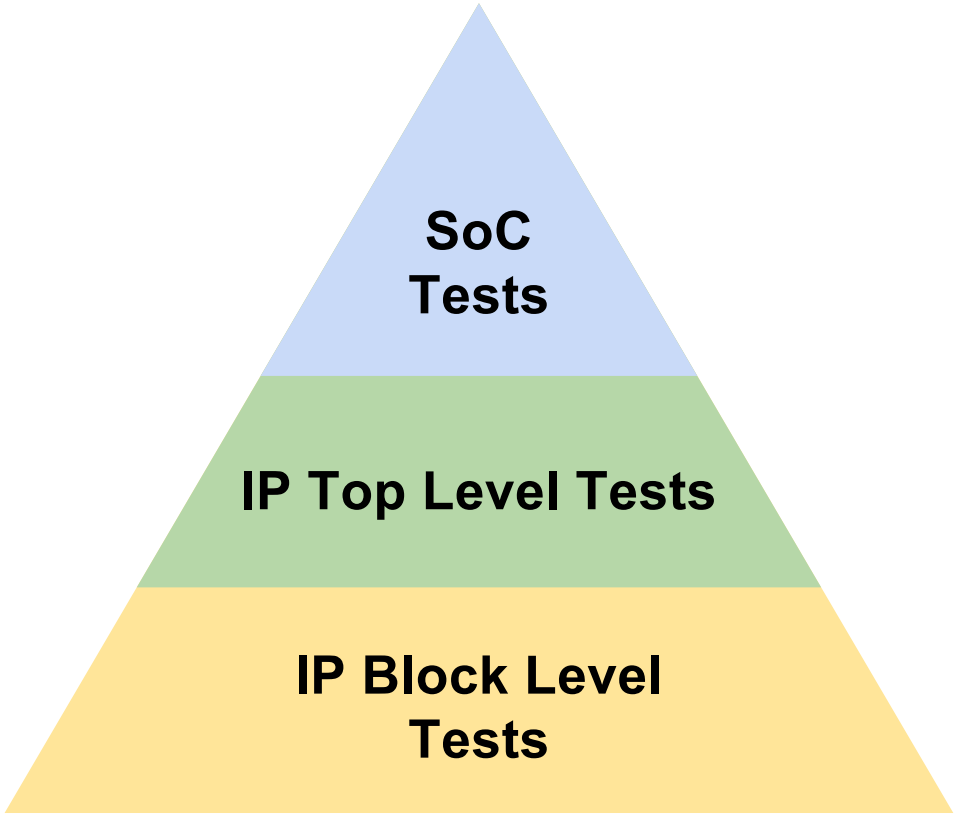


CTS

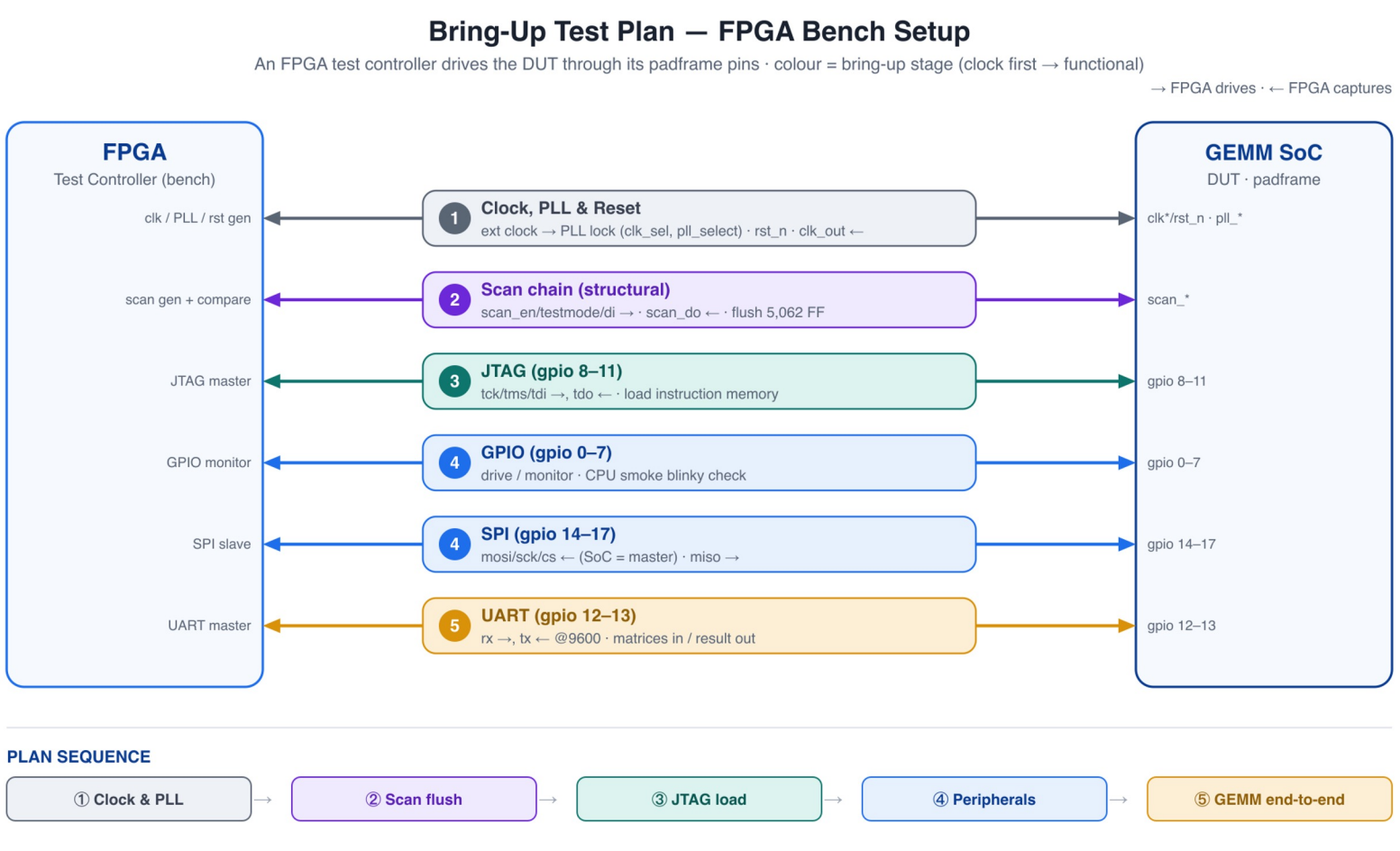

Clock Tree Debugger : corner_typ@ee-mill2.ee.ic.ac.uk
View Visibility Color by Help
cadence
Key
Control
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
0.45
0.50
0.55
0.60
0.65
0.70
0.75
0.80
0.85
0.90
0.95
1.00
1.05
1.10
1.15
1.20

Verification

- 3 tier verification
- All IP components verified independently
- Cocotb + external libraries to drive Wishbone, JTAG and UART interfaces
- ~90 unique test cases, all passing
- Random overnight regression testing



Bring-Up Test Plan



Reflections

Possible improvements:

- Architectural improvements:
  - By using more Systolic Arrays and adder-trees, we can achieve larger-scale matrix multiplication acceleration.
  - By adding vector-related units, we would be able to support Flash Attention.
- Verification improvements:
  - Coverage metrics
  - Formal verification
- Time management